

Numerical Analysis Assignment 2

nijatabbasov06

September 2024

Table of Contents

1	2.8	2
2	2.11	6
3	2.17	7
4	2.3.13	9

1

2.8

First, let us explain the reason why if the elements on the Diagonal matrix are powers of base of the floating point arithmetic, then we will not have a big rounding off error. The reason is that when we multiply by such a number or divide, then the new value is found by just right shift or left shift. This explains that, for example, if dividing number is 2^n , to represent new number, we only need to use n more digits in mantissa. But let us think about dividing 1 by 3. It clearly has to use all the digits in the mantissa, but for powers of the base of floating point arithmetic, dividing any number is guaranteed to just need n more digits to represent the new value.

In this problem, I auto-generated the Scaling, A matrix and the b vector. Because I generated a lot of them, I decided to find the standard deviation of the values on the diagonal of scaling matrix. And plot the relation between that and the condition number. Here, having a big standard deviation means my diagonal values are more skewed. And lower std means less skewed. Here is the following plot:

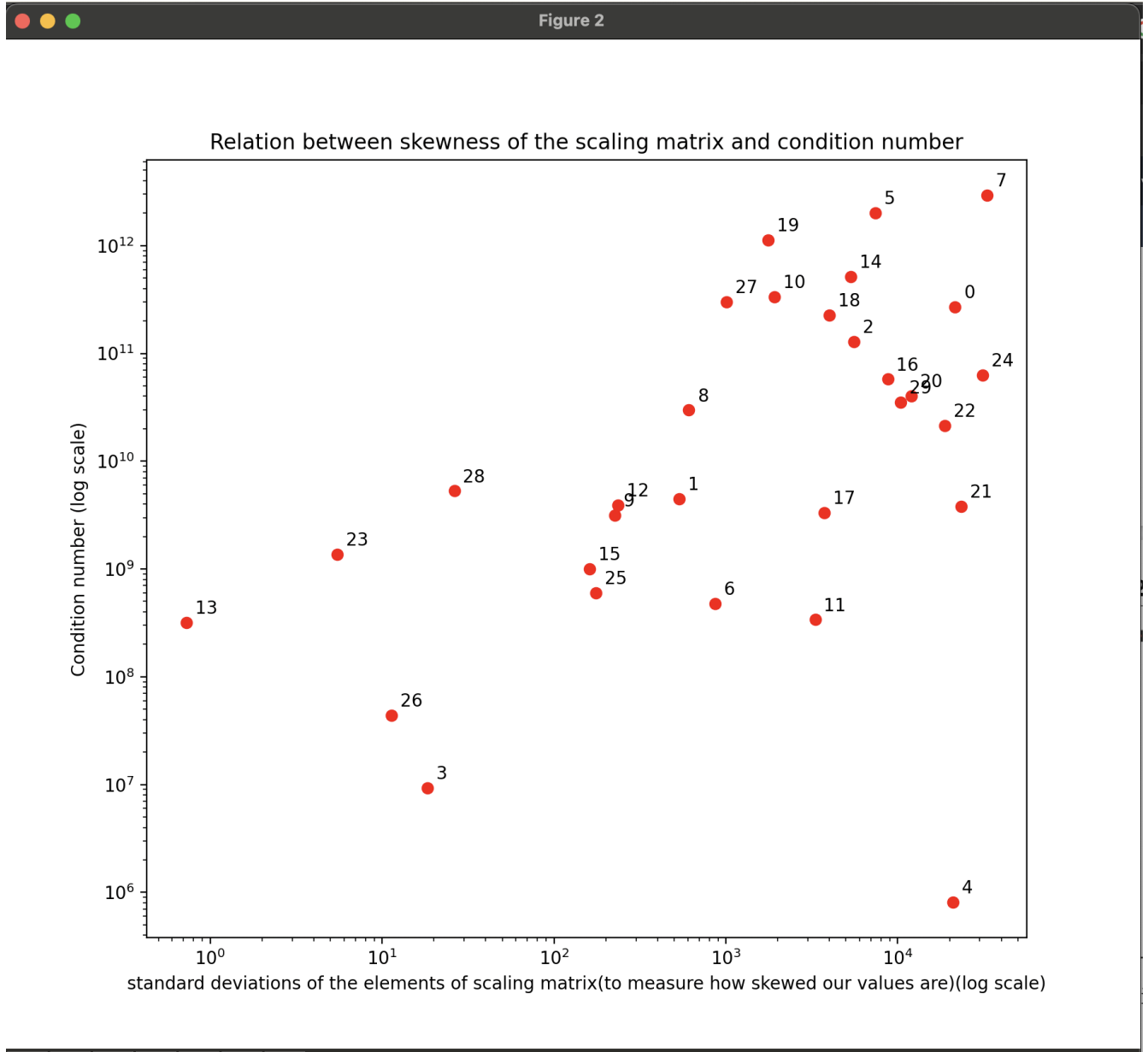


Figure 1: Condition number vs Standard deviation

We can see that here, for the lower Standard Deviation, there is very little number of matrices that result in big condition number. However, for the bigger values for Standard Deviation, Condition number are becoming bigger. This is reasonable, because $cond(DA) = \|DA\| \|A^{-1}D^{-1}\| \leq \|D\| \|D^{-1}\| \|A\| \|A^{-1}\| = cond(D)cond(A)$. We know $cond(D) = \frac{\max d_i}{\min d_i}$.

When the scaling matrix is more skewed, then $\text{cond}(D)$ becomes larger and this results in the trendline in the figure.

To explain how different scaling matrices affect the solution, I plotted the error of the solution with respect to the standard deviation of the scaling matrices:

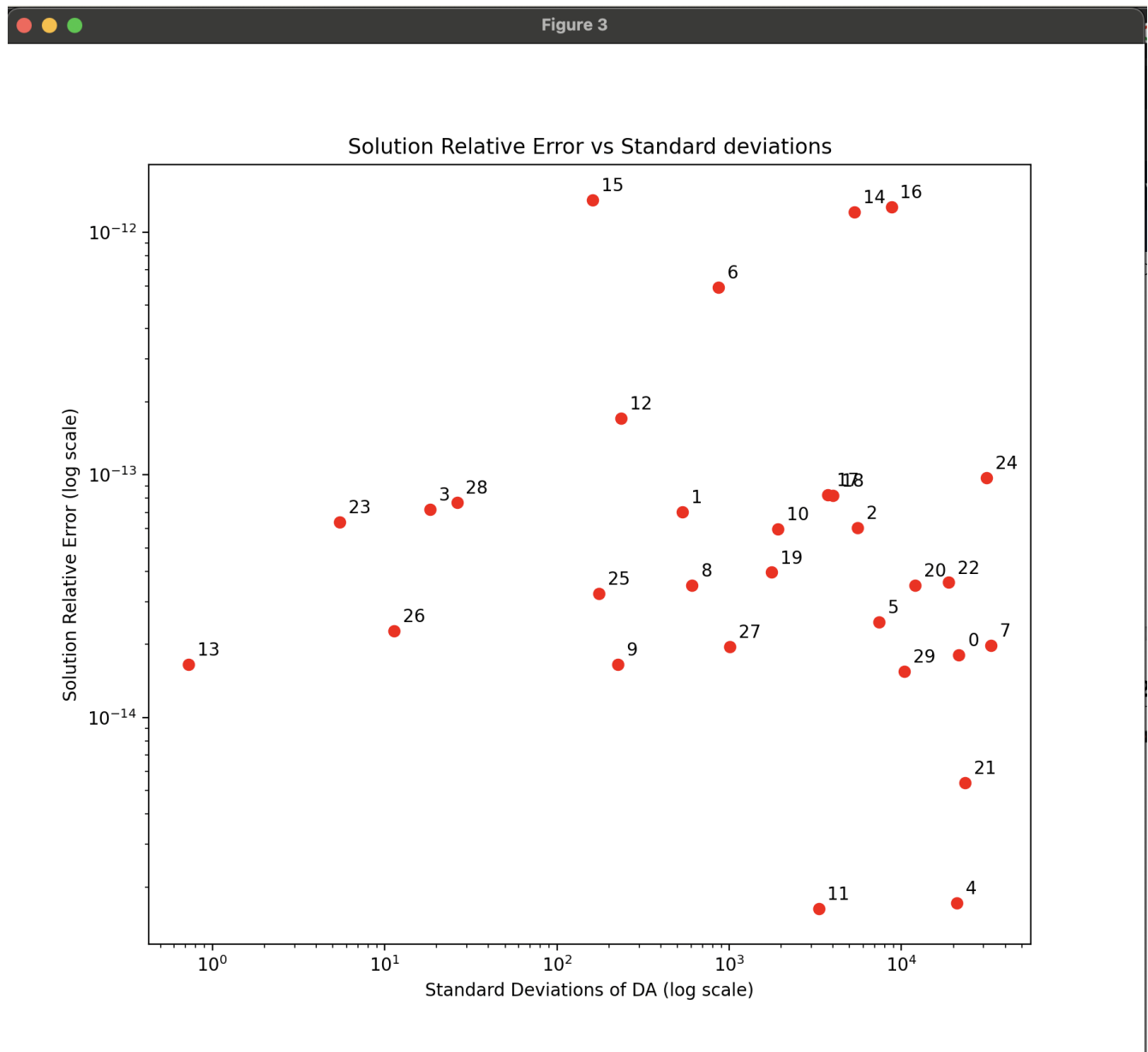


Figure 2: Solution Relative error vs standard deviation

In this plot, the effect of scaling matrixes with different stds seems random. It is probable that the scaling matrices which results in high errors have small values as their diagonal numbers.

The following shows the relation between solution error and the relative residuals

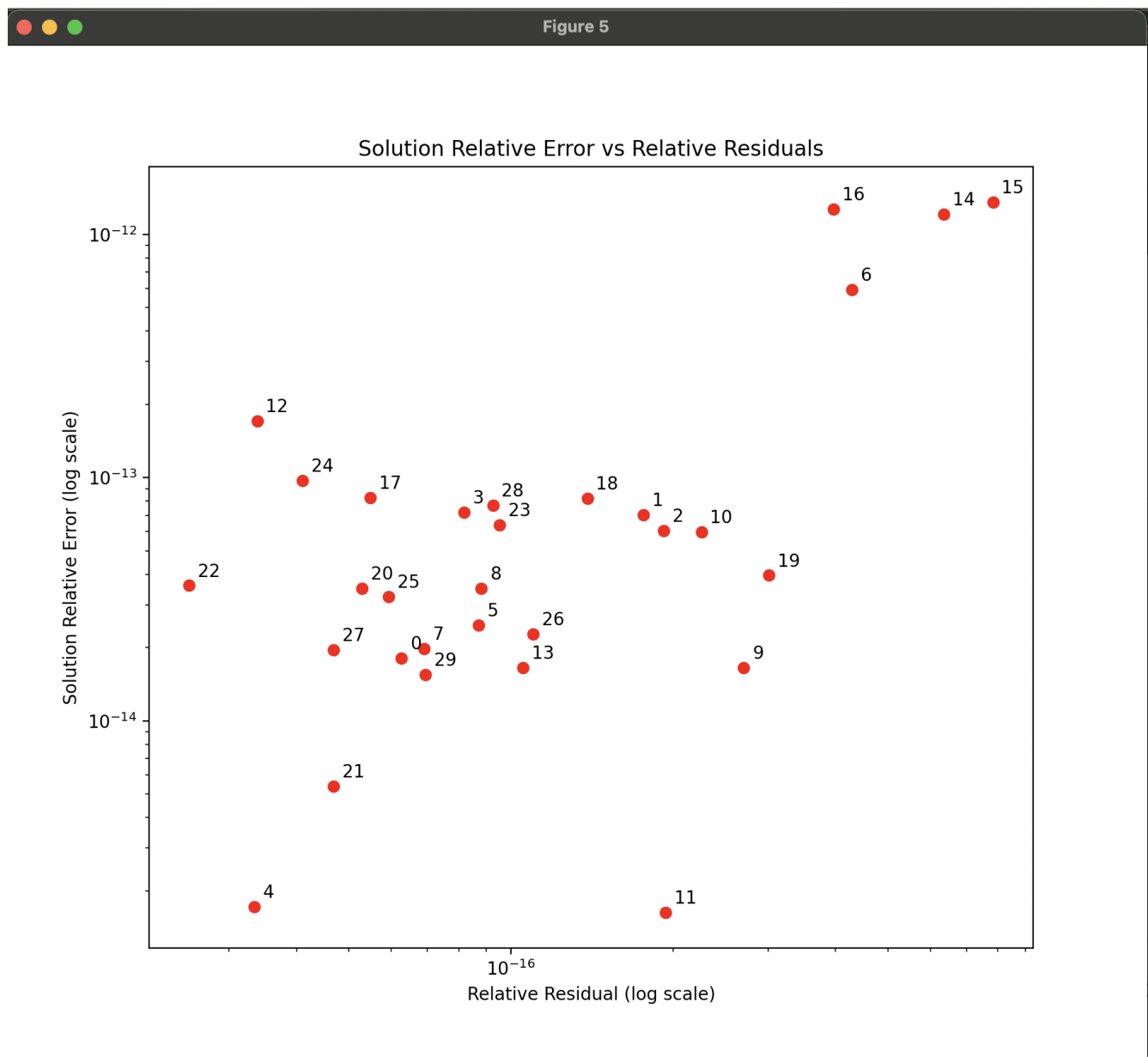


Figure 3: Solution Error vs Relative residuals

We can see that the graph follows a linear trendline, which is very reasonable because $\text{relative error} \leq \text{cond}(\text{DA}) * \text{relative residual}$.

But, we can see that in some points that has a low relative residual, the solution error can be really big. This is because those points have higher $\text{cond}(\text{DA})$.

The followings are the original matrix and the matrix that resulted in biggest error:

```
This is the Diagonal matrix that results in max error: [[3.11115961e-03 0.00000000e+00 0.00000000e+00 0.0000
0000e+00
0.00000000e+00]
[0.00000000e+00 1.27007460e-01 0.00000000e+00 0.00000000e+00
0.00000000e+00]
[0.00000000e+00 0.00000000e+00 3.59344817e+02 0.00000000e+00
0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 1.94080389e-03
0.00000000e+00]
[0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
5.88571323e-05]]
and its index is 15
Original matrix A:
[[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
[0.15599452 0.05808361 0.86617615 0.60111501 0.70807258]
[0.02058449 0.96990985 0.83244264 0.21233911 0.18182497]
[0.18340451 0.30424224 0.52475643 0.43194502 0.29122914]
[0.61185289 0.13949386 0.29214465 0.36636184 0.45606998]]
/opt/anaconda3/bin/python "/Users/nijatabbasov/Desktop/KAIST SEM 3/numerical analysis/assignment 2/problem_2.
8.py"
```

Figure 4: Scaling matrix with lowest accuracy

We can see that its index is 15. If we check the figure 3, it has the highest relative residual as expected.

2 2.11

The following are the results for pivot and no pivot cases:

Here we can see that the error for pivot is less than the no pivot case. Also, to not have bias, I ran the code for 7 different 50×50 matrices and took the average of the values such as error, residual, relative error of the matrices to represent the difference between two methods. In the no pivot case, we can see that the error is 12 times bigger than the pivot

```

AI51-SEM-3/numerical-analysis/assignment-2/problem_2.11.py

Without Pivot:
Error(avg): 1.2040527376196728e-11
Relative Error(avg): 2.9194871325769256e-12
Residual(avg): 2.1859195746526445e-12

Pivot:
Error(avg): 1.0032278551246756e-12
Relative Error(avg): 2.4570338949931906e-13
Residual(avg): 2.3130542633156952e-14
○ (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$

```

Figure 5: pivot vs no pivot

case. For the relative error, the no pivot one is 11.87 times bigger than the pivot case. Finally, the residual error of no pivot case is 94.63 times bigger than the pivot case

3 2.17

a) I used two method for solving the linear system, first was using the library for sparse matrices and the other was using a library for dense matrices. I checked if the solutions are close to each other by using `np.allclose()` function. The following is the time for each method.

We can see that Banded method is much more efficient timewise than the dense method.

Also, results are indeed same.

b) To verify if $A = RR^T$ I created R matrice and multiplied it by R^T . Then by using `np.allclose()`, I checked if the matrices are the same. We can see in figure 6 that, indeed $A = RR^T : True$ is written in the terminal.

The line "Solutions are close: True" means that the solution by using banded method is same with the solution from UL decomposition. I again checked if the answers are same by `np.allclose()`.

For my code, relative error for both methods were really small. But the

```

● (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$ /opt/anaconda3/bin/python "/U
ktop/KAIST SEM 3/numerical analysis/assignment 2/problem_2.17.py"
A = RR^T: True
Dense method time: 0.009845972061157227 seconds
Banded method time: 0.0002338886260986328 seconds
Solutions are close: True
To check if the solution from RR^T is equal to x_normal: True
Condition number of A: 1295546063031.7212
Relative error for banded method: 2.616593206290056e-06
Relative error for RR^T method: 1.3787624569006406e-06
Relative error after refinement (banded): 7.938600252109525e-06
Relative error after refinement (RR^T): 1.3787624569006406e-06
The RR^T method seems more accurate.
Iterative refinement did not significantly improve the accuracy of the banded meth
○ (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$

```

Figure 6: Result

RR^T method was more accurate than banded method. After refinement method applied to banded method result, our error become bigger. The reason for this can be that because error is already small, changing x — *for — banded* value by only a little bit results in the value to have minus error of the same value. Basically, it just keeps oscillating around the true value. In order to increase accuracy for this case, we might want to do refinement more times.

The relative error is always smaller or equal to the product of condition number and the relative perturbations error in the input. In this example, relative error for input perturbation is norm of machine epsilon vector divided by module of b . This can show the expected accuracy of our system

4 2.3.13

I ran the code for 50*1 dimensional vector.

Exact solution:

[illegible]

Absolute error:

```
[ 2.32611863e-08 -3.62339213e-06 1.27201699e-04 -1.54723603e-03
 3.40671113e-03 8.09653579e-02 -8.11425576e-01 3.41627292e+00
-7.00073604e+00 4.06367935e+00 9.95733955e+00 -1.70384314e+01
 1.56796998e+00 5.52180473e+00 1.28979048e+01 -8.78735367e+00
-1.38007630e+01 -2.02042704e+00 1.58323942e+01 8.71091757e+00
-1.00736648e+01 -5.06714289e+00 -6.86985757e+00 1.08236669e+01
-2.79945426e+00 1.01209127e+01 -1.70950388e+01 1.22811169e+01
 7.34968463e-01 -4.40128221e+00 -7.02676618e+00 1.56894176e+01
-4.44080229e+00 -2.29744909e+01 9.52512403e-01 3.46061789e+01
-1.30196241e+01 1.41582187e+00 4.24831361e+00 -1.42680709e+01
 1.68800178e+01 -2.31528348e+01 8.14503918e+00 -1.72623671e+01
 1.05607700e+01 2.05922783e+01 1.05928112e+01 -2.37590397e+01
-1.87054444e+00 3.87764685e-01 3.45729579e+00]
```

Condition number of A:

2.2229041143815e+19

```
○ (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$
```

Ln 35, Col 1 Spaces: 4 UTF-8 LF { } Pyth

Figure 7: Absolute error

I put all the errors in the Absolute error section.

For this time, I ran the code for 10 *1 dimensional vector:

```
[ 3.81865165e-06 -6.36158907e-04  2.64597134e-02 ...  7.99174611e+01
 -3.41911490e+02 -2.08860594e+02]
```

```
Condition number of A:
```

```
6.355730897863551e+20
```

```
● (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$ /opt/anaconda3/bin/python "/Use
ktop/KAIST SEM 3/numerical analysis/assignment 2/problem_2.3.13.py"
```

```
[1.          1.0000005  0.99998685  1.00014969  0.99909513  1.00321965
 0.99291948  1.00973466  0.9918551   1.00379217  0.99924677]
```

```
Computed solution c:
```

```
[1.          1.0000005  0.99998685  1.00014969  0.99909513  1.00321965
 0.99291948  1.00973466  0.9918551   1.00379217  0.99924677]
```

```
Exact solution:
```

```
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

```
Absolute error:
```

```
[-4.58501415e-09  4.95132777e-07 -1.31477051e-05  1.49692069e-04
 -9.04872761e-04  3.21965054e-03 -7.08052006e-03  9.73466046e-03
 -8.14489994e-03  3.79217489e-03 -7.53231583e-04]
```

```
Condition number of A:
```

```
521959222338558.8
```

```
○ (base) Nijats-MacBook-Pro:assignment 2 nijatabbasov$
```

```
Ln 20, Col 7  Spaces: 4  UTF-8  LF  { } Pyth
```

Figure 8: Absolute error

We can see that this time, the errors are much

We can see that matrix can be ill - conditioned because condition number is really high.